

הרצאה 5 – Javascript

בל הזכויות שמורות לקרן יעקב 2024



תוכן עניינים

1. מבוא כללי

2. תכונות

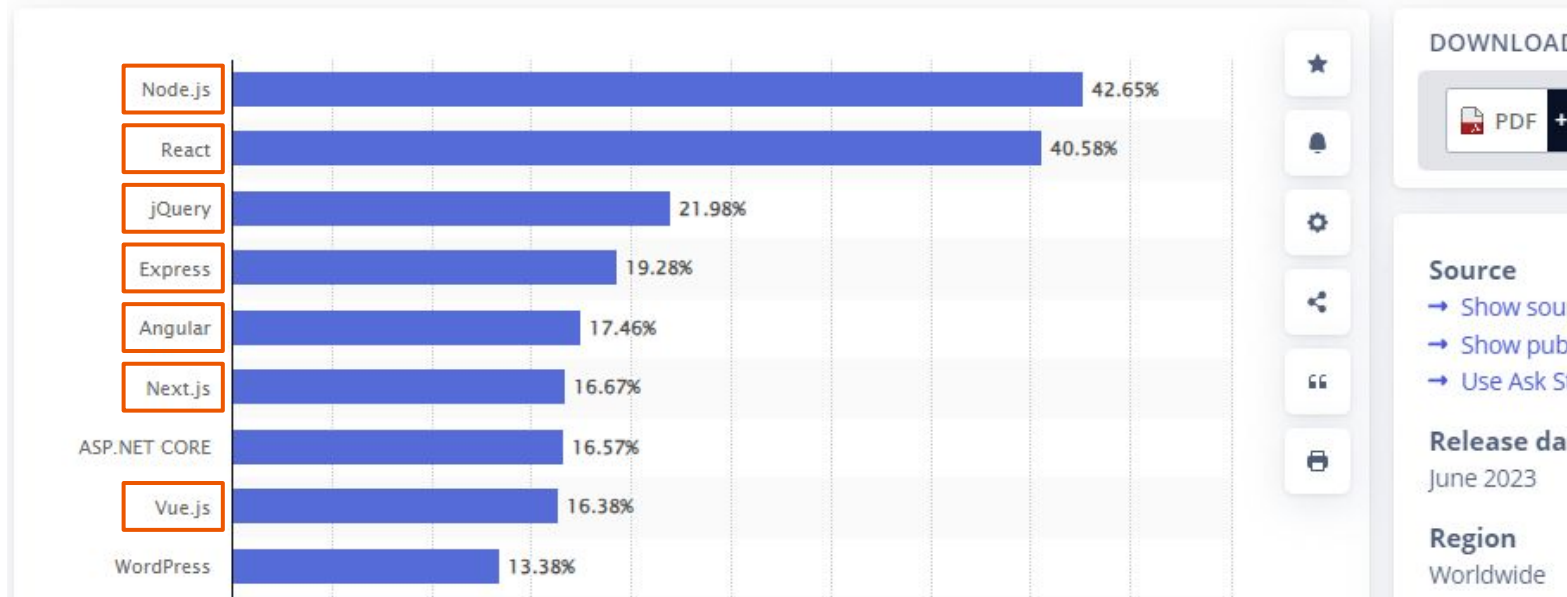
3. פונקציות



1. מבוא כללי

מוטיבציה

Most used web frameworks among developers worldwide, as of 2023



Javascript



- בקורס המבוא אנחנו מתמקדים בברזלים – "Vanilla Javascript"
- בקורס הבא (שירותי מארג האינטרנט) עולים ברמה –
 - לומדים JS הרבה יותר לעומק
 - בצד לקוח וגם בצד שרת, כאשר ההתמקדות תהיה בצד השרת
 - נבין ונתנסה בפרדיגמות התכנות Event-Driven ו- Functional Programming
 - נלמד REACT כחלק מ- MERN FULLSTACK

Javascript

- 
- JS תוכננה במקור להיות scripting language, שרץ ב-Web browser.
 - Javascript היא לא Java! זה לא שם קיצור.
 - כתובה בתקן [ECMAScript-262](#), הגרסה הנוכחית היא ES2023 או בשם אחר ES14
 - ההתפתחות העצומה של השפה עם השנים הובילה להרחבת השימוש בה. ממעברי עכבר ואימות נתוני טפסים, לשליטה ב- DOM, AJAX, ספריות כ- JQuery וקוד שרץ ללא דפדפן בצד השרת: Node.js.
 - להבדיל משפות תכנות אחרות שעוברות קומפילציה (הידור), שפת JavaScript נכתבת כטקסט, כמו HTML ו-CSS, ונקראת על ידי הדפדפן.
 - הדפדפן מקבל את ההוראות, סורק אותן מתחילתן ועד סופן ופועל על פי הקוד וההוראות
 - כשפת צד לקוח נרשמת בצירוף למסמך HTML ומתבצעת על גבי מחשב הלקוח
 - כשפת צד שרת נרשמת בתוך מסמך js ומתבצעת על מחשב השרת

הפרדת שכבות - חזרה והמשך

HTML

- שפה מבוססת תגיות (לא שפת תיכנות)
- במסמך HTML מגדירים את תבנית הדף עם משמעות ואת התוכן content (הטקסטים, התמונות, הקישורים) שמגיעים מה- DB
- בתגיות HTML לכל תגית יש תמיד סימני <...> מסביב לשם התגית

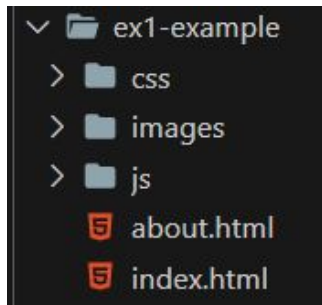
CSS

- שפה שמוכוונת על ידי רשימה של הגדרות עיצוב (לא שפת תיכנות)
- ב-CSS מגדירים את העיצוב של הדף, את אופן התצוגה presentation
- בסינטקס של CSS יש מסולסליים
- ניתן להחליף עיצוב על ידי שינוי הגדרות ב-CSS מבלי לשנות דבר בקוד HTML עצמו

JavaScript

- שפת תכנות
- בעזרתה מגדירים את ההתנהגות של האלמנטים

מבנה פרוייקט בקורס שלנו - חזרה



- הדף HTML הראשי בפרוייקט תמיד יהיה index.html
- דפי HTML נוספים יהיו ליד הדף הראשי
- קבצי תמונה יהיו בתוך תיקיית images
- קבצי עיצוב CSS יהיו בתוך תיקיית CSS
- קבצי javascript יהיו בתוך תיקיית js
- תיקיות נוספות תהיינה באותה היררכייה, במידת הצורך
- הכל בתוך תיקיית הפרוייקט.
- פרוייקטים אחרים יהיו בתיקות אחרות.

דרכים להרצת JS

• דרך ראשונה:

נכתוב את הקוד בתוך קובץ html וניתן לדפדפן להריץ את הקוד

מה הבעיה בשיטה זו?

```
1 <!DOCTYPE html>
2 <html>
3   <head>
4     <title>Javascript run 1</title>
5     <script>
6       let a = 6;
7       let b = 7;
8       window.alert(a + b);
9     </script>
10  </head>
11  <body>
12  </body>
13 </html>
```

127.0.0.1:5500 says

13

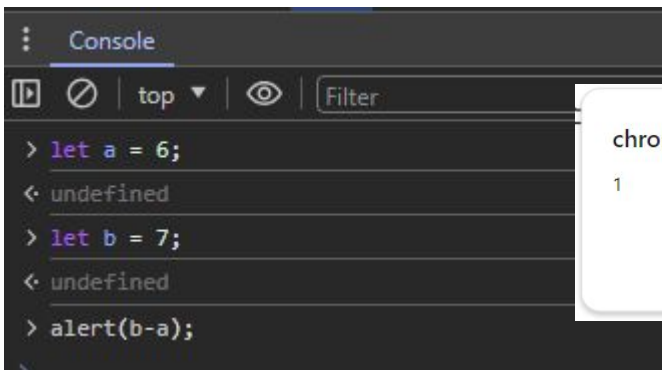
OK

דרכים להרצת JS - המשך

• דרך שנייה:

נכתוב את הקוד ישירות ב- console של הדפדפן (REPL- read evaluate print loop)

מה הבעיה בשיטה זו? מה קורה בשעושים רענון לדף?



```
> let a = 6;  
< undefined  
> let b = 7;  
< undefined  
> alert(b-a);
```

chrome://new-tab-page says

1

OK

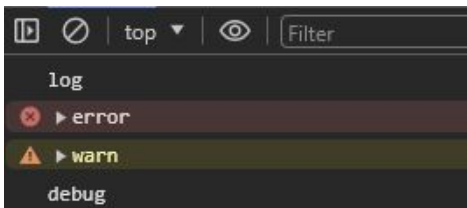
דרכים להרצת JS - המשך

• דרך שלישית:

נכתוב את הקוד בקובץ עם סיומת js ונצרף ל-head של מסמך ה-HTML

```
1 <!DOCTYPE html>
2 <html>
3   <head>
4     <title>1 - Javascript console</title>
5     <script src="js/scripts.js"></script>
6   </head>
7   <body>
8   </body>
9 </html>
```

```
1 console.log("log");
2 console.error("error");
3 console.warn("warn");
4 console.debug("debug");
```



ישנה שיטה רביעית - להריץ ע"י Node, נראה בהמשך הקורס

שתילת קוד JavaScript במסמך HTML – סיכום

- הדפדפן מפרש כטקסט כל תו שאינו תחום בתגית, ומציג אותו על המסך
- כדי להימנע מהצגת קוד JavaScript, שתחבירה אינו כולל סוגריים משולשים, יש לדווח לדפדפן על המעבר מ-HTML ל-JavaScript
- הגדרת אזור ה-JavaScript נעשה על ידי התגיות script, בדר"כ ב-head
- בין התגיות אפשר לרשום את הקוד JS עצמו או להגדיר attribute בשם src ולציין את הנתיב לקובץ ה-JS

```
1 <!DOCTYPE html>
2 <html>
3   <head>
4     <title>1 - Javascript console</title>
5     <script src="js/scripts.js"> </script>
6   </head>
7   <body>
8   </body>
9 </html>
```

```
1 <!DOCTYPE html>
2 <html>
3   <head>
4     <title>1 - Javascript console</title>
5     <script>
6       console.log("Hello World!");
7     </script>
8   </head>
9   <body>
10  </body>
11 </html>
```



2. תכונות

Javascript

- חלק מההוראות הן ברמת תצוגה, וחלקן ברמת חישובים, הקצאת משאבי זיכרון, מערכים, יצירת לולאות ועוד. סוג זה של תכנות נקרא client side programming.
- JS יכול להיכתב בפרדיגמות התכנות Functional Programming ו- Event-Driven
- JS אינה שפה מבוססת עצמים Object Oriented based, היא מבוססת פונקציות המוגדרות כאובייקטים
- ההורשה ב- JS מבוצעת על ידי prototyping ולא Classes
- משתמשים בתחביר הנקודה dot syntax כמו student.year
 - מגדירים משתנים גלובליים ולוקאליים בתוך אובייקטים
 - פונקציה היא גם אובייקט
 - לאובייקטים יש שיטות methods. שיטות הן פעולות השייכות לאובייקט
 - לאובייקט יש אירועים events. אירוע המשתייך לדוגמא לאובייקט קישור היא מעבר עכבר

Welcome to Javascript



אובייקט object

- האובייקט הוא המרכיב הבסיסי של השפה, הוא אוסף של מאפיינים, אירועים, משתנים, שיטות
- אובייקטים של שפת JavaScript כגון: אובייקט תאריך (Date), אובייקט מתמטיקה (Math)
- אפשר גם ליצור אובייקטים בעצמנו

```
const person = {  
  firstName: "John",  
  lastName: "Doe",  
  age: 50,  
  eyeColor: "blue"  
};
```

- אובייקט המסמך document – מתאר את מסמך ה-HTML ולכן הוא מכיל את תכונות המסמך
○ כגון, צבע רקע: document.backgroundColor

Hoisting

- לפי ההתנהגות הדיפולטית של Javascript – העברת כל ההצהרות לתחילת הסקופ, גם אם הן פיזית לא נמצאות שם. רק הצהרות נקראות קודם, לא ההשמות (גם אם הן ביחד)
- JS בעצמו מניף (Hoisting) את כל ההצהרות על המשתנים ועל פונקציות (בלי לקרוא להן) לראש הדף (או לראש ה-scope) כדי לשמור מקום בזיכרון, כאשר החוק אינו תקף להשמות.
- משתנים המוגדרים ע"י let, const גם נקראים קודם אבל לא מאותחלים, המשמעות היא שהקוד מודע למשתנה, אבל אם ננסה להשתמש בו נקבל שגיאת Reference Error. המשתנה var לעומת זאת (שלא נשתמש בו בקורס) יקבל את הערך undefined
- יתר הקוד נקרא לפי סדר הופעתו במסמך

```
console.log (greeter);  
var greeter = "say hello"
```

it is interpreted as this:

```
var greeter;  
console.log(greeter); // greeter is undefined  
greeter = "say hello"
```

Scopes & Variables

- block הוא איזור התחום בסוגריים מסולסלים (אין קשר לבלוק של CSS)

- ההצהרה **var** מאפשרת להגדיר משתנים גלובלים, כך ש-block של קוד לא משפיע עליהם

```
{  
  var x = 2;  
}  
// x CAN be used here
```

With **var** you **can** do this:

```
var x = "John Doe";  
  
var x = 0;
```

- ההצהרות **let** או **const** מאפשרות להגדיר משתנה ל-block של קוד.

```
{  
  let x = 2;  
}  
// x can NOT be used here
```

With **let** you **can not** do this:

```
let x = "John Doe";  
  
let x = 0;
```

לכן לא נשתמש בהצהרות עם **var** !

שיטה method



```
let str = 'Hello World';  
let length = str.length;  
  
console.log(length); // 11
```

```
let arr = ['apple', 'banana', 'cherry'];  
arr.push('orange');  
  
console.log(arr); // ['apple', 'banana', 'cherry', 'orange']
```

- עושים הפרדה בין פונקציה לבין מתודה.
- מתודה היא פונקציה המשוייכת לאובייקט מסוים



3. פונקציות

פונקציות



- js היא functional programming
- פונקציות הן ערכים שאפשר לשמור במשתנים או להחזיר מהן ערך, לשלוח אותן כמשתנה או להחזיר בערך
- ישנם מספר דרכים לכתוב פונקציות ב-Javascript, הן לא דווקא סותרות אחת את השנייה
 1. Function Declaration
 2. Function Expression
 3. פונקציית חץ - Arrow Function
 4. פונקציה אנונימית
 5. IIFE - פונקציה המפעילה את עצמה

1. Function Declaration

```
function average(x, y) {  
  let a = x + y;  
}
```

```
function sumIt(a, b) {  
  return a + b;  
}
```

- הצהרה על פונקציה נשמרת בזכרון לשימוש עתידי, דוגמאות:
 - הפונקציה הבאה אינה מחזירה ערך והיא נטענת לזיכרון בהתחלה

- הפונקציה הבאה מחזירה ערך והיא נטענת לזיכרון בהתחלה

- מתבצע פה hoisting – ההצהרות על הפונקציות מועלות לראש הדף

- הפונקציות נקראות עם סוגריים, לדוגמה: sumIt()

2. Function Expression

- אנו יכולים לעשות השמה של פונקציה למשתנה
- פונקציות שמוגדרות כך מופעלות מיד ואינן מוקפצות לראש הדף – **hoisting** אין

```
let sumIt = function getIt(a, b) {  
  return a + b;  
};
```

- הפונקציה הבאה נקראת ונטענת לזכרון מיד כשהקוד מגיע לשורה זו (זו שורת פקודה ולכן בסופה יש נקודה פסיק)

המשתנה `sumIt` מחזיק רפרנס לפונקציה ולא רק את הערך שחוזר
בשנרצה לקרוא לפונקציה נעשה זאת ע"י `sumIt(1, 2)`;

מאחר ולא צריך את השם של הפונקציה `getIt`, אפשר לרשום את הפונקציה כך:

```
let sumIt = function (a, b) {  
  return a + b;  
};
```

3. פונקציית חץ

- מאחר ו-JS היא שפה מבוססת פונקציות ואובייקטים, וכותבים function הרבה מאוד פעמים, החל מגרסה ES6 פיתחו סינטקס שמאפשר לוותר על המילה function, ולהשתמש ב-**שווה פרמטרים חץ**

```
30 let sumIt = (a, b) => {  
31   |   return a + b;  
32   | };  
33  
34 let returnIt = a => {  
35   |   console.log(a);  
36   |   return a;  
37   | };  
38  
39 let verifyIt = () => {  
40   |   console.log('in verifyIt');  
41   | };  
42  
43 sumIt(1,2);  
44 returnIt(sumIt(1,2));  
45 verifyIt();
```


4. פונקציה אנונימית



- פונקציה ללא שם
- אי אפשר לגשת אליה לאחר האיתחול (אי אפשר לקרוא לה ללא שם...) ולכן לרוב נעשה לה השמה למשתנה

```
let show = function() {  
  console.log('Anonymous function');  
};  
  
show();
```

```
let add = (a, b) => a + b;
```

- אפשר גם להשתמש בה כ-IIFE (בעמוד הבא) או כמשתנה לפונקציה אחרת

```
setTimeout(function() {  
  console.log('Execute later after 1 second')  
}, 1000);
```

5. IIFE

IIFE – Immediately-invoked Function Expression

- פונקציה שמפעילה את עצמה ברגע שהיא נקראת. כלומר זו פונקציה שמופעלת מיידית, פעם אחת, לא צריכה טריגר חיצוני (לא צריך לקרוא לה) ונכתבת בסוף ה-body.
- הסינטקס הוא פונקציה בתוך סוגריים (שיוצר expression) ולאחר מכן סוגריים ריקים בשביל ההפעלה עצמה.

This is the syntax that defines an IIFE:

```
(function() {  
    /* */  
})();
```

IIFEs can be defined with arrow functions as well:

```
((() => {  
    /* */  
}))()
```

5. IIFE (1)

```
8 // IIFE as arrow function example
9 (() => {
10     console.log("Hello World");
11 })();
12
13 // IIFE as anonymous function example
14 (function () {
15     console.log("Hello World");
16 })();
```

- דוגמאות לפונקציות המפעילות את עצמן

5. IIFE (2)

- דוגמה לפונקציה המשתמשת ב-IIFE

```
6 <body>
7   <article>This layer will be hidden</article>
8   <button id="toggle">Show / Hide Hidden Layer</button>
9   <script>
10    (function(){
11      document.getElementById("toggle").onclick = function(){
12        let article = document.getElementsByTagName("article")[0];
13        if(article.style.display == "none"){
14          article.style.display = "block";
15        }
16        else{
17          article.style.display = "none";
18        }
19      };
20    })();
21  </script>
22 </body>
```

This layer will be hidden

Show / Hide Hidden Layer

Show / Hide Hidden Layer